Project: Serverless & Microservice Patterns

Due: 02/28/2026 11:59 PM
Extended Deadline: 03/01/2026 11:59 PM

Introduction
- Objectives
- Context & Prerequisites
- Evaluation Method
- Handout

knative
- Task Overview
- Task

grpc
- Task Overview
- Task

dapr
- Task Overview
- Task

Wrap Up
- Conclusion
- End of project survey

dapr (Task)

Moving from Single-Service Solutions to Multi-Service Solutions

In the previous task we focused on grpc to improve the latency of our fastAPI LLM server.

You have demonstrated to your boss that **it is possible** to make the server faster, but he then comes back and tells you the solution is not sufficient for your clients:

Task 2's solution, gRPC, **can scale**, but **its interface will not work with clients that can use gRPC**. The frontend team is adamant that they will **only** work with REST-compliant APIs, and that **they will not use gRPC in the frontend**.

Given this, it looks like a **single service solution** is impossible. Thus, we shall focus on **architecting multiple services together** to accomplish our goal.

System Diagram

At the end of this task, you will have a system architecture similar to the diagram shown below:

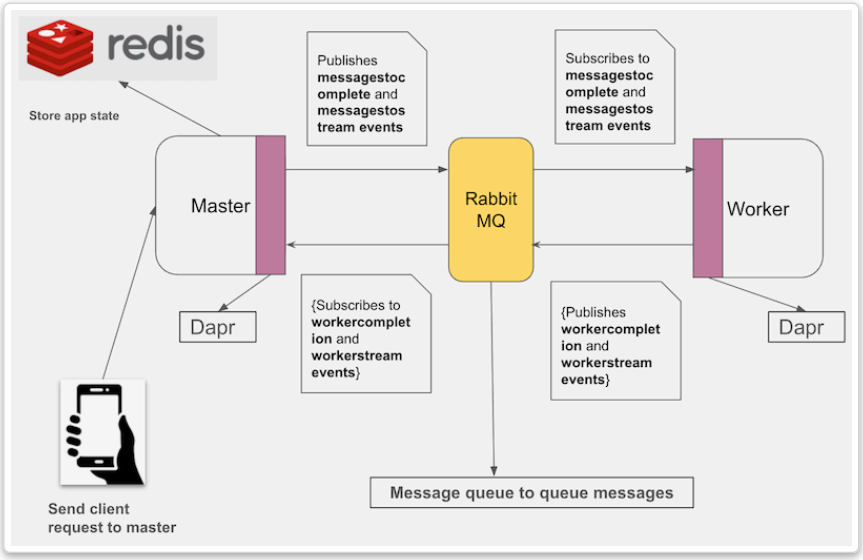


Figure: Dapr Architecture Diagram

We shall have **two services:**

1. A **worker service** that takes our requests and sends **streams of completions** back.
2. A **master service** that just **acts** like a nice REST-Friendly interface for your frontend.

The second service will **ensure that you have a low-latency** system, while **still allowing you** to **pretend** that the client is seeing updates real-time. The client can then **repeatedly request** or **poll** the second service to get updates when it wants to.

To do this, we will need a **message queue**.

Message queues are simple **buffers** that hold **messages** so that they can be processed. Think of them like **the API gateway** in the last project, except **only focused on slowing down** requests as opposed to anything else. Their main benefit is that **they allow you to standardize processing of asynchronous messages**, as opposed to just **synchronous messages**.

The issue with **message queues** is that, in general, **each one has a different protocol**, and while we could **have you implement each and every async protocol**, there is not **enough** there to make it worth your time to learn to do so.

Instead, we will **use a managed service** to help with our networking issues. In this task, we will use **Dapr**, the **Distributed Application Runtime**. It is an in-between from **implementing your own code** to **handling all of networking for you** that was created by **Microsoft** for the purposes of making **distributed systems** easier to use, and is **very different from Istio**, our **service mesh** which we will use in the future projects.

Unlike Istio, **Dapr** does not care about **routing** requests and **splitting** traffic to make streams more readable. Instead, it is more focused on providing a **developer-friendly** way to **architect microservice apps**.

It does this through a **sidecar**. This is a **container** that runs alongside **your main container**, and helps handle communication issues. You mostly focus on **your application**, and let **the sidecar** handle the gory details.

> **Information**
>
> Dapr, in many ways, works like a more **developer-friendly** Kubernetes. While we shall use it as it is an excellent tool, remember that you **can** do a lot of your implementation without it, and only introduce it when things start to get more complicated and you want a more managed solution.

Dapr Setup

Dapr comes in two parts - the **Dapr CLI** and the **Dapr Runtime**:

The **Dapr CLI** primarily serves as how you will **interact** with Dapr, both locally and on **Kubernetes**. It provides fairly easy abstractions, and serves as a **kind of replacement** for **kubectl**.

> **Information**
>
> In most cases, you will still want to use **kubectl** for debugging your deployment and other issues. While the CLI will be convenient for local debugging, you know by this point that local debugging and kubernetes debugging are fairly different beasts, with the latter being significantly harder to do.

On your studentvm, please start by running the following to install **Dapr's CLI**:

```
wget -q https://raw.githubusercontent.com/dapr/cli/master/install/ins
```

You will need an admin password for this command to work properly. You can look into your VM's settings to add one if you haven't.

Once that is done, we will **initialize Dapr locally** through the following command:

```
dapr init
```

This will setup the **Dapr runtime** locally onto our system. This will allow us to work and test our system **locally** before deploying it through Kubernetes.

> **Information**
>
> In this class, we generally like to **work locally as much as possible** before deploying. It can simplify some of the work necessary to make things easy.
>
> It is important to note that this may not be possible. You will need to pay careful attention to ports as your application scales, as a single VM can only support so many ports among all of the services you can implement.

Dapr - In Practice

We discussed earlier that **Dapr** uses **sidecars** in order to simplify network trafffic. Let's first unpack this into some more detail before going and trying to use it.

Dapr is a fairly large system, and it contains a ton of **building-blocks** which try to simplify several developer-facing challenges common to cloud native applications, some of which are even **beyond just communication**. This includes:

Service invocation, like that which Kubernetes provides, through its own **service discovery** system.
Simple state management, State can be any data your service requires. Services can also generate data and this data has to be stored somewhere and Dapr does that in a more developer-friendly way.
Pub-sub, allowing for easy utilization of **message queues**.
Bindings, which allow you to **mesh Dapper-related services** with everything else.
Actors, which can allow for potentially easier interactive service development using a more intuitive object-oriented style approach.
Workflows, which can enable you to abstract away some aspects of service development using higher-order services.
Secrets, which can provide Kubernetes-like **secret** management to your applications.
Distributed Locking, which can enable forced-consistency among data processing.
Cryptography, which can enable encryption and decryption of data without it being part of the application itself.

As you can see, Dapr has a **lot** going on with it. It tries to handle most, if not all, of the cross cutting concerns you would have when developing microservices, but instead of acting as a **separate service**, it acts like an interceptor of existing traffic to enable easier developer scaling.

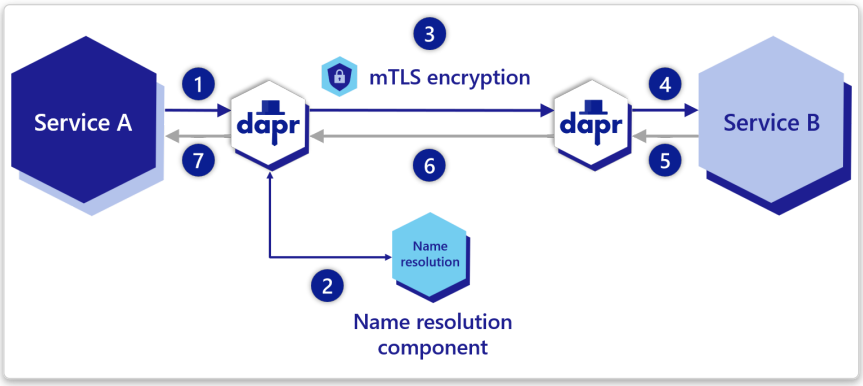
For our purposes, we will only teach you how to use the **service invocation**, **state management**, and **pub-sub** components for this task. They contain enough for us to dig into to make it worthwhile, while also ensuring that the task is not too overwhelming.

Let's go over the **service invocation** endpoint before diving deeper into what we need to do for this task.

Dapr Service Invocation

The easiest of the building blocks we will first use is Dapr's **Service Invocation** block. This block uses the **Dapr sidecar** to handle communication for your container automatically.

The following figure (taken directly from Dapr's documentation), shows a high-level process for how Dapr handles service to service communication:



1. Service A sends a message to the **Dapr** sidecar. This can be either **HTTP** or **gRPC**, and is **not encrypted**.
2. The Dapr sidecar talks to the Dapr **name resolution component** to get the **address** of Service B's sidecar.
3. Service A's sidecar then sends the message to Service B. This is done over gRPC serialization for the sake of performance, but **is encrypted with mTLS**.
4. Service B's sidecar sends it to Service **B** for processing.

In practice, this just means that **Dapr** handles most of the issues of getting it to work for you - you just need to worry about **sending your messages to the sidecar**.

Subtask 1: Dapr-izing your FastAPI Solution

Information

This subtask is meant to be on the easier side.

Create a `dapr` folder in the `project3` folder of the handout alongside grpc-p4 folder, run `cd dapr` to navigate into dapr folder and then run `poetry new worker` to create the `worker` app. This app will handle the worker instance using our previous `llm_server` FastAPI code.

Create a new file and name it *worker_simple_st4.py* in the directory *./dapr/worker/src/worker*.

You must copy over the code (from ./llm_server/llm_server/main.py) written for your previous project and use it as your starter code.

You would need to update the "model_path" in your worker_simple_st4.py file since the model is located in the `llm_server` folder.

New path:

```
model_path="../../../llm_server/tinyllama-1.1b-chat-v1.0.Q2_K.gguf"
```

Also copy over some content from **pyproject.toml** file from the previous project (from ./llm_server/pyproject.toml) as a guide to port over the packages we need in ./dapr/worker/pyproject.toml. Take note that we're referring to the outer `worker` directory. To simplify things:

1. There's already an existing `pyproject.toml` file in the `dapr/worker/` directory. Just copy the important stuff from the previous project's `pyproject.toml` file.
2. You should mainly focus on the dependencies your new app needs to run. Both app and dev dependencies.

Once that is done, you made your first Dapr app!

To run it, first make sure you're in the `project3/dapr/worker` directory and run the following command:

```
poetry install
```

Under the `dapr/worker` directory, run the following command to start the server:

```
dapr run --app-port 8001 --app-id llm-worker --app-protocol http --da
```

Danger

You may run into an error that reads like the following:

"instruction crashed due to server illegal instruction (core dumped)"

A workaround is to run the following command:
`CMAKE_ARGS="-DLLAMA_AVX2=OFF -DLLAMA_FMA=OFF -DLLAMA_F16C=OFF" FORCE_CMAKE=1 pip install llama-cpp-python --no-cache-dir --force-reinstall`

This does look a bit nasty, but let's walk through the arguments just so that you understand what's going on, from beginning to end:

`dapr run` is how you will run **Dapr** apps. We need to use it so that **Dapr** knows that we are working with its sidecar as opposed to just the native app.
`--app-port` specifies the port that your local app will run on. In our case, this is just **8001**, but you could choose whatever port you like here. If this is wrong, the **Dapr** sidecar will be unable to reach your app.
`--app-protocol` specifies what protocol your app uses internally. By default, this is **http** (note the lack of the "S" here), but you could also use **gRPC**.

`--dapr-http-port` specifies the port that's open on the **Dapr sidecar**. For local development, each of these ports will have to be **different**, but on **Kubernetes** they can be the same to make your life easier.
`--` is an empty flag. It's used by command line applications like Dapr to say "there are no other flags here", and it allows you to add your command in full without your terminal asking if **dapr** has the arguments.
Out of our **poetry run** command, which should be familiar to you, just focus on the `--port` part. Note that it is **8001**. That needs to be the same as `--app-port`'s value.

With your command running, let's test that it works.

Open a new **terminal**, and run:

```
curl http://localhost:3501/v1.0/invoke/llm-worker/method/api?message=
```

You should get a response that at this point looks really similar to what you have worked with over the past few weeks:

```
{"id":"cmpl-23a4ed81-24cf-46fb-9a30-dd3db3d2e4ba","object":"text_comp
```

Here, we **reached our application** using the **Dapr sidecar process**. While the URL can look a bit scary, let's break it down a bit:

`v1.0` is just the API version number. It won't change for our application, but you can set it.
`invoke` tells the **Dapr sidecar** that you want to run an app that is hosted on it.
`llm-worker` is our **app's** `app-id`. This tells our **Dapr sidecar** that we want to use the `llm-worker` app.
`method` tells the **Dapr sidecar** you want to run a specific method on the application.
`api?message=hi` tells **Dapr** what method you want to use. `api`, and the arguments for that method.

As you can hopefully see, **Dapr** is not as scary as the **FastAPI** app we have. While we still have a fair amount to code, keep in mind that, at its core, **Dapr** is meant to be developer-friendly.

Interim Validation

Make sure that you can get an output from the above command. As you shouldn't need to change any of your existing code, there should not be too much of an issue.

Subtask 2: Pub-Sub Eventing for a Stream of Data

Let's start by working on the **worker** service. Our first goal will be to **connect** our existing API to a **RabbitMQ** message queue using **Dapr**.

To get started, use **poetry add** to get the following dependencies:

dapr
dapr-ext-fastapi

These will provide the **Dapr SDK** to your application, which will make it much easier to work with your **message queue**.

In order to add a **message queue** to a Dapr application, we need two separate things:

1. An existing **container** with the **message queue** running.
2. A Dapr **component file**, that explains to Dapr what and where our message queue is.

If we were running our system in Kubernetes, the first task would be as simple as **using a RabbitMQ helm chart**. However, as we will stick to local development for this question, we will instead need to use **Docker** to run **a local RabbitMQ process**. To do so, run the following:

```
docker run -d -p 5672:5672 --name rabbitmq rabbitmq:3-management-alpi
```

The rabbitmq container will exit once you stop your VM. If you decide to stop your VM and continue working on the project later on, you need to start rabbitmq again. To do this run

the following command: docker start rabbitmq You won't get Dapr to subscribe and send messages if rabbitmq is down. See example error message below: Could not update sidecar metadata for cliPID: PUT http://127.0.0.1:33685/v1.0/metadata/cliPID giving up after 5 attempt(s): Put "http://127.0.0.1:33685/v1.0/metadata/cliPID": dial tcp 127.0.0.1:33685: connect: connection refused

This will create and run a **rabbitmq** message queue as a separate docker process, using port **5672** for communication.

After installing the queue, we need to handle task 2, which is to tell **Dapr** where our queue is, and how to use it. The easiest way to do this is to **create a component yaml**. This is a file, like a **Kubernetes YAML**, that specifically tells the **Dapr sidecar** where and how to interact with our existing system.

Create a **dapr-components** directory in the `dapr` directory. Then create a file named **pubsub.yaml** following this example. You will only need to change the **scopes** section to refer to the **app-ids** we will give our final services - **llmmaster** and **llmworker**, and the name to **pubsub**. Here, the scopes specifically refer to which **Dapr services** are actually allowed to use the message queue; without them, you will be unable to connect to the queue with both the worker and the master.

With that file created, let's now **modify** our application so that it **subscribes** to our **message queue**. Luckily, with the **Dapr FastAPI Extension**, this is fairly easy. Here's an example of a simple application that subscribes to the **workermessages** topic, and prints out whatever it gets to the console:

```
from dapr.ext.fastapi import DaprApp
from fastapi import FastAPI, Depends, Query
from pydantic import BaseModel
from typing import Literal

app = FastAPI()
dapp = DaprApp(app)

class SimpleCloudEvent(BaseModel):
    data: str
    datacontenttype: str
    id: str
    pubsubname: str
    source: str
    specversion: str
    topic: str
    traceid: str
    traceparent: str
    tracestate: str
    type: str

@dapp.subscribe(pubsub="pubsub", topic="workermessages")
def print_test_message(message: SimpleCloudEvent) -> Literal["OK"]:
    print(message.data)
    return "OK"
```

Let's discuss some of the details of what is going on before you start trying to get something working:

1. The best way to think about a **topic** is that it is like a **separate queue**. Any message that comes to **workermessages** will end up at **at least one** pod that subscribes to the **workermessages** topic, as that is the guarantee that you get when using Dapr.
2. The message type **has to be some variation of SimpleCloudEvent**. Dapr, by default, sends messages in the CloudEvent standard, which is just an industry standard for messaging that plays well with existing logging and tracing tools. Here, our input is a string, so we can just make **data** a string, but you can make **data** any other Pydantic

model, such as **BaseLlamaResponse**, and **FastAPI** will handle the conversion and validation for you.
3. We generally recommend returning something after getting the topic. This generally helps the queue know that your system has **processed** the message. While we will **not** use that feature for this project, it is still best-practice.

If you want to run this example, simply create a `baseworker.py` python file in your **src/worker** folder, and use something like **dapr run --app-port 8003 --app-id llmmaster --resources-path ../dapr-components -- poetry run uvicorn worker.baseworker:app --host 0.0.0.0 --port 8003** to start running it.

Once you have it running, make sure that you see the line:

```
== APP == INFO:     127.0.0.1:40094 - "GET /dapr/subscribe HTTP/1.1"
```

This just indicates that **Dapr** can find our **rabbitmq** queue. To test that this is working, we can use **Dapr's CLI** to send a test message. In another terminal, run the following:

```
dapr publish --publish-app-id llmmaster --pubsub pubsub --topic worke
```

You should see the following appear in the terminal that you are running your app in:

```
== APP == Test Message
== APP == INFO:     127.0.0.1:39822 - "POST /events/pubsub/workermess
```

If you do, then congratulations! You are now ready to start developing the worker side of your application. Your task is to:

1. Implement your own Dapr app in the `dapr/worker/src/worker_simple_st4.py` file.
2. Add a simple event that **subscribes to the** `messagestocomplete` topic, and when it recieves any **SimpleCloudEvent** message, does the following:
   a. Validates that the token count in **data** is as expected.
   b. Prints the response our **llm-service** will have. We will eventually change this to send a message to the queue, but this is an easier thing to get started with.
3. Add a second listener that **subscribes to the** `messagestostream` topic, and when it recieves any **SimpleCloudEvent** message, does the following:
   a. Validates that the token count in **data** is as expected.
   b. Uses something like **llm(template, temperature=0.0, stream=True)** to get a **generator** of **partial completions** for our llm. You can do this with a for loop, like:

```
for elem in llm(template, temperature=0.0, stream=True):
    print(elem)
```

c. For each element in that generator, **prints out the element** to console.

Interim Validation

Before proceeding, test that sending a **message** to each of the topics causes your LLMWorker to print out the messages we expect. This will be vital before we start modifying your service and getting it to **send** messages as well.

Subtask 3: Sending Messages in our Worker

Now that we have a system that **can take messages**, let's modify it to send **messages** as well. This is also **quite easy** to do with the Dapr SDK.

As a simple example, here's some code that **publishes some messages programmatically to the workermessages** topic from earlier:

```
from dapr.clients import DaprClient
import json

with DaprClient() as client:
```

```
message_to_send = {"key1":"test1", "key2":"test2"}
client.publish_event(pubsub_name="pubsub", topic_name="workermess
```

As you can see, there is not that much going on:

1. The `DaprClient() as client` line opens a **gRPC** connection between our system and the **Dapr sidecar**, which it will use to send the message to our message queue.
2. The `client.publish_event` line will then send our message.

What's important to realize is that **client.publish_event** can only send **strings**. Thus, you will need to take any **dictionary** you want to send and use **json.dumps** to convert it into a **string** before sending. The **data_content_type** is just a field that helps **Dapr** realize that you are sending a **JSON** object.

For this subtask, we want you to modify your existing **llmworker** system to **send messages** as opposed to **just recieving them**. However, before we do that, let's discuss something important to think about before we just blindly go and make this happen - **indempotency**.

Indempotency and Message Ordering

A big challenge that comes when you want to send a **stream** of messages is that, as you scale your system, it **becomes harder and harder** to ensure that **messages** are sent as expected. In general, when you use a **message queue**, you want to design your system to be tolerant in two ways:

1. **Message recievers should be indempotent**, so if the message broker retries your message the response is the same.
2. **Messages can be handled in any order**, so even if you send a particular set of messages in some order, it is important to make sure that you **make it clear to the message recipient** what the original order was if it matters.

However, our messages as-is make it hard for **a recipient** to utilize. We would like to ensure the following:

1. It should be clear what **message** our worker sends relates to what **message** our master asks it to complete.
2. It should be clear what **part of the message** our worker sends relates to what **message** our master provides.

If we look at what our current system will try to send:

```
== APP == Sending message: {'id': 'cmpl-b11eaecf-1928-475e-ad1d-72aac
== APP == Sending message: {'id': 'cmpl-b11eaecf-1928-475e-ad1d-72aac
== APP == Sending message: {'id': 'cmpl-b11eaecf-1928-475e-ad1d-72aac
== APP == Sending message: {'id': 'cmpl-b11eaecf-1928-475e-ad1d-72aac
```

We can see that:

1. The **existing completion id** is useless, as it is not clear **what the original message was**.
2. There is **no message order** present within the existing output of **llama-cpp-python**. If we got the messages in some random order, it would be incredibly difficult to know what the expected index is, and moreover what **token** any completion refers to.

For these reasons, **you need to modify the completion and streaming messages** so that:

1. The **id** field in any message we send must be an **md5 hash** of our existing message. This will make it easy for us to **relate the original message** with **the completion**. To get the **md5** hash of a string, here's a simple function:

Code:

```
import hashlib

def get_hash(message:str) -> str:
    return hashlib.md5(message.encode('utf8')).hexdigest()

print(get_hash("This is a test"))
```

Run

Output:

1. You must add a **message_idx** field to your **streaming message**, where the **idx** field is just the **generation index** of your message. To get this index, we recommend using something like the **enumerate** function, which takes in a generator and converts it into a generator that gives the **element** in the generator along with the index. Again, here's a simple example to help make it clearer what we mean:

Code:

```
listme = [4,5,6,7,9]

for idx, elem in enumerate(listme):
    print(idx)
    print(elem)
```

Run

Output:

Interim Validation

With that said, your goal for this part is to have a system that:

1. When it receives a message from the "messagestocomplete" topic:
    1. Gets the data from the message, the string we want our LLM to respond to.
    2. Gets the LLM's response for that message.
    3. Changes the **id** field to use an **md5 hash** of the **message** we want to complete.
    4. Publishes the output to the **workercompletionevents** topic as a **json**.
2. When it receives a message from the "messagestostream" topic:
    1. Loops over the **streaming output** for our generator.
    2. Modifies the **id** and **message_idx** fields of our output as we discuss above.
    3. Sends it to the **workerstreamevents** topic.

We recommend **printing the output** of your system **before** sending the message, as this will generally make it easier to **debug** once we get to the **master** service.

To test, feel free to run the app using Dapr and **use Dapr publish** to send a message.

If you use something like this:

```
dapr publish --publish-app-id llmmaster --pubsub pubsub --topic messa
```

We expect the last few lines of the output of your system to look like:

```
== APP == Sending message: {'id': 'd1d4180b7e411c4be86b00fb2ee103eb',
== APP == Sending message: {'id': 'd1d4180b7e411c4be86b00fb2ee103eb',
== APP == Sending message: {'id': 'd1d4180b7e411c4be86b00fb2ee103eb',
```

Similarly, if you use something like this to send a message to your **messagestocomplete topic**:

```
dapr publish --publish-app-id llmmaster --pubsub pubsub --topic messa
```

We expect something like the following as a response:

```
== APP == Sending message: {'id': 'd1d4180b7e411c4be86b00fb2ee103eb',
```

Note here that the **id** fields should match exactly. We are not testing the **message** in any detail due to the nature of LLMs, but we still expect the message indices to behave similarly.

Subtask 4: Creating a Master Service

Now that we have a streaming system, we need to create a **master** service that:

1. Can agglomerate the data from our streaming app.
2. Can send it to a database for easy processing.
3. Can be easily queried by the user.

While this sounds **fairly hard**, Dapr will make this fairly painless, so let's start writing it up. `cd` into the **dapr** directory and run `poetry new` to create a new app named "master" for this system.

Add the following dependencies using `poetry add`:

```
fastapi
uvicorn
gunicorn
dapr
```

As a reminder, our worker system **expects** messages on the following topics:

1. messagestocomplete
2. messagestostream

and sends messages to the following topics:

1. workercompletionevents
2. workerstreamevents

Conversely, our master service will **expect** messages on the following topics:

1. workercompletionevents
2. workerstreamevents

and send messages to the following topics:

1. messagestocomplete
2. messagestostream

> **Information**
>
> This is not necessarily the only way we could architect the communication protocol for our system. We could, for example, have a single topic for both **completion** and **streaming** responses.
>
> However, this is the easiest way to do it to make our business logic the tiniest bit easier.

Thus, we will need at least **2** message handlers for our system - one for **messagestocomplete** and one for **messagestostream**. We will also need a **/api** route that will act as the **message sender** for our **Master service**.

> **Information**
>
> This all is fairly involved, so we will guide you through it. In general, apps with state, even the kind we talk about here, are **fairly involved**, so be prepared.

Subtask 4.1: The /api Route and State-Stores

Let's start off with the main route that external clients will use - our **/api** route. We know that we want a **state-store** so that we can **ensure** that our master process knows **the current state** of any message.

We will need to create another **Dapr Component** for that. Dapr, by default, provides a **Redis** docker container for this purpose in **local installations**, so please add the following

to **dapr-components/statestore.yaml** so that we can make use of it:

```
apiVersion: dapr.io/v1alpha1
kind: Component
metadata:
  name: statestore
spec:
  type: state.redis
  version: v1
  metadata:
  - name: redisHost
    value: localhost:6379
  - name: redisPassword
    value: ""
  - name: actorStateStore
    value: "true"
```

All this does is **link** the Dapr sidecar to the Redis docker container, and give it the name **statestore** so that we can use it. Redis, unlike the databases you have worked with in Data Engineering, is a **really simple** database, and only has support for **key** and **value** pairs - you give it a **key**, and it will get you the **value** that it has stored in the location.

Create a file and name it **master_st4.py** in the directory **dapr/master/src/master**.

For this service, our **key** will be the **md5 hash** of our message, as that is what we used for our worker service. Our value, however, will be a dictionary with three keys:

- `message_status`, which is either **Starting**, **Processing**, or **Done**.
- `completion_type`, which is either `completion` or `streaming`.
- `message`, which will contain our message.

Thus, for the **/api** route, define a function **get_message_status** with parameters `message: str` and `completion_type : Literal["completion", "streaming"]`. Here, the **Literal** type is something you will need to import from **typing**, but is used when you have a set of **values** your data must be in. It's an easier-to-use version of an Enum, if you have worked with that type before.

This route must do the following:

1. Compute the **id** of the message using the **md5** hash function we showed you earlier.
2. Get the value associated with that key from the redis state store. Here's a simple example that gets the value and prints it:

   ```
   with DaprClient() as client:
       id = hashlib.md5(message.encode('utf8')).hexdigest()
       result = client.get_state("statestore", id).data
       print(result)
   ```

   Here, the **.data** field always contains the value associated with the result of calling **client.get_state**.
3. If the **result** is empty (has a length of 0), then we first must **initialize** the value with an easy default:

   ```
   default_state = {
       "message_status":"Starting",
       "completion_type":completion_type,
       "message":""
   }
   ```

   To set the state for some key, you can use something like the following:

   ```
   with DaprClient() as client:
       client.save_state("statestore", id, json.dumps(default_state))
   ```

4. After setting the default state, we must also:
   send a **message** to our worker so that it can start working on the **LLM task**. Here, the **message** is just our **message** parameter, but the topic will depend on **completion_type** - it is either **messagestocomplete** or **messagestostream**.
   Use the `client.publish_event(pubsub_name="pubsub", topic_name=topic, data=message, data_content_type="text/plain")`.
   Note that we're sending the message in plain text format.
   Still in this `if` block, you should return a **response** object with the following fields:
   - `id`: the id you computed
   - `message_status` should say **Starting**
   - `message` should be an empty string
5. Now, we enter the else part. If the **result** is not empty, then we must use **json.loads** on **result.data** to get our **current value in our state store**. For the **message** key, we will either use a **string** or a **list** when we have messages we want to understand. If the **completion_type** for the message was **completion**, then the **message** field will just be a string that corresponds to our message. Otherwise, our **message** will be a **JSON-encoded list** of the tokens we have gotten. We can convert this into our **current message** using `''.join(json_loaded_value["message"])`.
6. After this, you must return a dictionary with the following:
   1. **id**, the md5-encoded id of our message.
   2. **message_status**, which we get from our state-store.
   3. **message**, which must not be a list but a string.

To make your life easier, here's a Pydantic model for our response:

```
class WaiterResponse(BaseModel):
    id: str = "my_cool_id"
    message_status: Literal["Starting", "Processing", "Done"] = "Star
    message: Optional[str] = None
```

Interim Validation

Check that this works by first running our system with:

```
dapr run --app-port 8003 --app-id llmmaster --resources-path ../dapr-
```

Note that you have to exit any previous running instance to free up the port 8003.

You can then **curl** your service using:

```
curl "http://localhost:3500/v1.0/invoke/llmmaster/method/api?message=
```

The output should be:

```
{"id":"098f6bcd4621d373cade4e832627b4f6","message_status":"Starting",
```

Subtask 4.2: The subscriber routes

Now that you have a simple **getter** to query your API, let's work on the message-broker related routes. To make your life **a bit easier**, we will give you the **workerstreamevents** handler, but you will need to write the **workercompletionevents** handler yourself.

Here's the workerstreamevents handler in question:

```
@dapp.subscribe(pubsub='pubsub', topic='workerstreamevents')
def add_stream_response(message: CloudEventStreamLlamaResponse) -> No
    with DaprClient() as client:
        message = message.data
        primary_id = message.id
        interim_result = json.loads(client.get_state("statestore", pr
        if len(interim_result) == 0:
            interim_result = [""]*(message.message_idx + 1)
            interim_result[message.message_idx] = message.choices[0].
```

```
        else:
            if message.message_idx >= len(interim_result):
                interim_result.extend([""]*(message.message_idx + 1 -
            interim_result[message.message_idx] = message.choices[0].
        current_state = json.loads(client.get_state("statestore", pri
        current_state["message"] = interim_result
        current_state["message_status"] = "Processing" if message.cho
        client.save_state("statestore", primary_id, json.dumps(curren
```

Where **CloudEventStreamLlamaResponse** uses the following model:

```
class CloudEventStreamLlamaResponse(BaseModel):
    data: StreamLLamaResponse
    datacontenttype: str
    id: str
    pubsubname: str
    source: str
    specversion: str
    topic: str
    traceid: str
    traceparent: str
    tracestate: str
    type: str
```

and **StreamLlamaResponse** is a modified version of our **BaseLlamaResponse** object:

```
class StreamLLamaResponse(BaseModel):
    id: str = "cmpl-7fc1be4c-8f5b-4b2f-805f-f8c5086a9fb4"
    message_idx: int = 0
    object: str = "text_completion"
    created: int = 1708459650
    model: str = "tinyllama-1.1b-chat-v1.0.Q2_K.gguf"
    choices: List[Choice]
```

Let's quickly walk through what the code is doing, so that you understand what's going on, as it is pretty cool:

1. When we get an event, we first get the message's id field, which corresponds to our **primary key** for our database - the **md5 hash** of our message.
2. When then load the **value** associated with that key, to get **interim_result**, a list of tokens that have been generated.
3. If the **interim_result** list is **empty**, then we know that it doesn't exist. As we do not know **ahead of time** what the size of our completion is, we create a list of **empty strings** that is of size **message.message_idx + 1**, which is just the smallest size that will ensure that we can load it as expected. We then **add the token** we just got to that list, at the **message_idx** location.
4. If the **interim_result** list is **not_empty**, then either the list is large enough to fit our **new token** or it is not. If it is not, we **extend** the list to contain new empty strings so that our new **message_idx** token works, and then **add our new token** to the list.
5. Afterwards, we just use **json.loads** to convert the element from a list to a string, store it into our **message** field, and then use **save_state** to save that value to our system.

You do not need to understand all of this, but what is important to realize is that **when you work with systems that are not-indempotent by default**, you need to start writing logic like this to ensure that your state makes sense.

With that said, you must now write the handler for when there is a **message** on topic **workercompletionevents**. All this needs to do is the following:

1. Get the primary key for our message.
2. Get the existing state.

3. Replace the message field with the LLM Worker's result.
4. Replace message_status with "Done"
5. Save the state to the state store.

> **Information**
>
> Generally, this is about 5-7 lines.

Interim Validation

You can check on the values of your application by sending messages with **dapr publish** and using the **/api** route to get the **current value** in any state. You should make sure that the **streaming** and **completion** event handlers work as you expect, as we imagine debugging during this next step will be a pain.

Subtask 5: Putting It All Together

Now that you have a worker, and a potentially reliable master service, all you need to do is to **put them together**. So far, we have only talked about **running individual apps in Dapr locally**. For this task, we will need to run **multiple apps at once**. The easiest way to do this is with a **dapr-multirun** yaml file.

Under the **dapr** folder, please create a file named `dapr_multiapp.yaml` with the following contents:

```
version: 1
common: # optional section for variables shared across apps
  resourcesPath: ./dapr-components
  daprdLogDestination: fileAndConsole
  appLogDestination: fileAndConsole
apps:
  - appID: llmmaster
    appDirPath: master/
    appPort: 8001
    daprHTTPPort: 3510
    command: ["poetry", "run", "uvicorn", "master.master_st4:app", "-
  - appID: llmworker
    appDirPath: worker
    appPort: 8002
    daprHTTPPort: 3511
    command: ["poetry", "run", "uvicorn", "worker.worker_simple_st4:a
```

All this does is very similar to the **single app** runs you have, but all of the configuration is done **in a YAML** instead of as command line arguments. For our purposes, you do not need to really understand what's going on here, but it should be fairly reasonable. The only thing that we want to point out are the **daprdLogDestination** and **appLogDestination** lines, as by default **dapr's multirun** doesn't give you both the **Dapr logs** and the **app logs** when you run it.

Final Validation

Assuming you have completed everything thus far, you should be set to test that your app works by first running it with:

```
dapr run -f dapr_multiapp.yaml
```

You can then either **go to localhost:8001/docs** or use curl to test that you can get streaming output.

We expect that there will be bugs you will need to figure out and handle at this point - here are a few things that should help you debug:

> By default, **rabbitmq** keeps any messages you send and **redis** keeps any elements stored within its state **across multiple calls to dapr run**. This can make it difficult to debug if you need to reset your **state store and message broker**.

To reset them, first use **docker container list** to get the container **ids** for both:

```
CONTAINER ID   IMAGE                          COMMAND
e862214975cb   rabbitmq:3-management-alpine   "docker-entrypoint.s…"
34fd98f86fcd   openzipkin/zipkin              "start-zipkin"
7ca511c85cd3   redis:6                        "docker-entrypoint.s…"
5db3a25a9faa   daprio/dapr:1.12.5             "./placement"
```

Then, you can run a command like **docker restart e862214975cb** for rabbitmq, and **docker run --rm -it --link 7ca511c85cd3 redis:6 redis-cli -h 7ca511c85cd3 flushall** for redis to clear the database.

> If you get into strange **Dapr** sidecar errors, simply restart your deployment. Sometimes Dapr can get into a weird state where the gRPC client breaks, and the easiest way to fix this is to restart your containers and re-run.
> You will probably get a fair amount of validation errors, as **Pydantic** will complain about the input and output for what you send and recieve. The easiest way to resolve these errors is to make your input type **str** and then print the dictionary, as the **Pydantic error message** you will get will be rather hard to read.

Submission

Tree Structure for this Task

```
dapr
├── dapr-components
│   ├── pubsub.yaml
│   └── statestore.yaml
├── dapr_multiapp.yaml
├── master
│   ├── README.md
│   ├── poetry.lock
│   ├── poetry.toml
│   ├── pyproject.toml
│   ├── src
│   │   └── master
│   │       ├── __init__.py
│   │       └── master_st4.py
│   └── tests
│       └── __init__.py
└── worker
    ├── README.md
    ├── poetry.lock
    ├── poetry.toml
    ├── pyproject.toml
    ├── src
    │   └── worker
    │       ├── __init__.py
    │       ├── baseworker.py
    │       └── worker_simple_st4.py
    └── tests
        └── __init__.py
```

Before submitting,

> please make sure that your **Dapr process** is running on the ports we expect, and that **you have reset the RabbitMQ and Redis** instances accordingly.
> In your `.gitignore` file, add `.dapr`. The submitter generates log files which we do not want you to push to git.

Then run

```
./submitter task3
```

Post Task Activity

Once you receive your desired scores, make a pull request from your feature branch to the main branch. After which you should merge your changes.

← Previous Page Next Page →

Report an Issue? © 2026 Copyright Sail() Platform